LAPORAN PRAKTIKUM
STRUKTUR DATA
IMPLEMENTASI *ABSTRACT DATA TYPE (ADT) STACK* MENGGUNAKAN *ARRAY* DAN *ARRAYLIST* DALAM *JAVA*

Disusun Oleh:

Haliya Isma Husna Putri Ahmadi
2511532002

Dosen Pengampu:
Wahyudi. Dr. S.T.M.T

Asisten Praktikum:
Muhammad Galid

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan Syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas Rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum Struktur Data pekan 3 dengan judul "Implementasi *Abstract Data Type (ADT)* Stack Menggunakan *Array* dan *ArrayList* dalam Pemrograman *Java*" tepat pada waktunya.

Penyusunan laporan ini bertujuan untuk memenuhi salah satu tugas dalam mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran bagi penulis dalam memahami konsep *Abstract Data Type (ADT) Stack* serta penerapannya dalam pemrograman Java. Melalui praktikum ini, penulis mempelajari bagaimana mengelola data menggunakan prinsip *LIFO (Last In First Out*) dengan menerapkan konsep *ADT*, serta mengimplementasikan operasi seperti *push*, *pop*, *peek*, dan *isEmpty*. Selain itu, penulis juga mengimplementasikan *Stack* menggunakan *Array* dan *ArrayList*, serta menerapkannya dalam beberapa studi kasus seperti pencarian nilai maksimum dan evaluasi ekspresi *postfit*.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat terbuka terhadap kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Ucapan terima kasih yang sebesar-besarnya penulis sampaikan kepada dosen pengampu, asisten praktikum, serta semua pihak yang terlibat dalam pelaksanaan praktikum dan penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi penulis maupun pembaca.

Padang, 13 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR ii

DAFTAR ISI iii

BAB I PENDAHULUAN 5

1.1 Latar Belakang 5

1.2 Tujuan 6

1.3 Manfaat 6

BAB II PEMBAHASAN 7

2.1 Dasar Teori 7

2.1.1 Abstract Data Type (ADT). 7

2.1.2 Konsep Kerja Abstract Data Type (ADT) 7

2.1.3* Komponen dalam *Abstract Data Type (ADT) 8

2.1.4 Primitif Type* dalam *Abstract Data Type (ADT) 9

2.1.5 *Stack* 10

2.1.6 Implementasi *Stack* dalam *Java* 11

2.2 Langkah Kerja 12

2.2.1 Pembuatan *Package* dan *Class* 12

2.2.2 Membuat Program *StackArray*_2511532002 16

2.2.3 Membuat Program *StackArrayDriver*_2511532002 18

2.2.4 Membuat Program latihanStack_2511532002 19

2.2.5 Membuat Program NilaiMaksimum_2511532002 20

2.2.6 SiswaStack_2511532002 22

2.2.7 Membuat Program *StackPostfix_2511532002* 24

2.2.8 *Upload* ke GitHub 26

2.3 Hasil **28**

2.3.1 Hasil *StackArrayDriver*_2511532002 ..... **28**

2.3.2* Hasil l*atihanStack_2511532002 ..... **29**

2.3.3* Hasil *NilaiMaksimum_2511532002 ..... **29**

2.3.4* Hasil *SiswaStack_251152002 ..... **29**

2.3.5* Hasil *StackPostfix_2511532002 ..... **30**

BAB III KESIMPULAN **31**

3.1 Kesimpulan **31**

3.2 Saran **31**

DAFTAR PUSTAKA **32**

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan pemrograman, pengolahan data yang terstruktur sangat penting untuk menghasilkan program yang efisien, terorganisir, dan mudah dikembangkan. Salah satu konsep yang digunakan dalam struktur data adalah *Abstract Data Type (ADT*), yaitu suatu tipe data abstrak yang mendefinisikan data beserta operasi-operasi yang dapat dilakukan terhadap data tersebut tanpa memperhatikan implementasi internalnya.

Salah satu bentuk *ADT* yang sering digunakan adalah Stack, yaitu struktur data yang menggunakan prinsip *Last In First Out (LIFO),* di mana elemen terakhir yang dimasukkan akan menjadi elemen pertama yang dikeluarkan. *Stack* banyak digunakan dalam berbagai kasus seperti pengelolaan data sementara, evaluasi ekspresi matematika, serta proses pemanggilan fungsi dalam program.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami cara merancang dan mengimplementasikan *Abstract Data Type (ADT) Stack* menggunakan bahasa pemrograman *Java*. Implementasi dilakukan dengan berbagai pendekatan, seperti menggunakan *Array, ArrayList*, dan *library* bawaan *Java*. Selain itu, mahasiswa juga mempelajari operasi-operasi dasar pada *Stack* seperti *push, pop, peek,* dan *isEmpty*, serta penerapannya dalam penyelesaian masalah seperti pencarian nilai maksimum dan evaluasi ekspresi *postfi*

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1.2.1 Memahami konsep dasar *Abstract Data Type (ADT)* dalam struktur data.

1.2.2 Mampu mengimplementasikan *Abstract Data Type (ADT)* menggunakan *Array, ArrayList, library* dalam bahasa pemrograman *Java.*

1.2.3 Mampu mengimplementasikan operasi dasar *Stack* seperti *push, pop, peek,* dan *isEmpty.*

1.2.4 Mampu menerapkan *Stack* dalam penyelesaian masalah seperti pencarian nilai maksimum dan evaluasi ekspresi *postfix*

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum ini antara lain:

1.3.1 Menambah pemahaman mengenai konsep *ADT Stack* dalam pemrograman.

1.3.2 Meningkatkan kemampuan dalam mengimplementasikan struktur data menggunakan berbagai metode.

1.3.3 Membantu dalam memahami penerapan *Stack* dalam penyelesaian masalah komputasi.

1.3.4 Menjadi dasar dalam pengembangan program yang lebih kompleks di masa yang akan datang.

1.3.5 Menjadi dasar dalam pengembangan program yang lebih kompleks di masa yang akan datang.

BAB II
PEMBAHASAN

2.1 Dasar Teori

Struktur data merupakan salah satu konsep penting dalam ilmu komputer yang digunakan untuk mengorganisasi dan menyimpan data agar dapat diakses dan diolah secara efisien. Dalam pengembangan program, diperlukan suatu cara untuk merepresentasikan data beserta operasi-operasi yang dapat dilakukan terhadapnya secara terstruktur. Salah satu pendekatan yang digunakan adalah *Abstract Data Type (ADT).*

ADT memungkinkan programmer untuk fokus pada fungsi dan perilaku suatu data tanpa harus langsung memikirkan bagaimana data tersebut diimplementasikan di dalam memori. Dengan demikian, penggunaan *ADT* dapat meningkatkan modularitas, keterbacaan, serta kemudahan dalam pengembangan dan pemeliharaan program nantinya.

2.1.1 Abstract Data Type (ADT).

Abstract Data Type (ADT) merupakan suatu konsep yang mendefinisikan sebuah tipe data beserta sekumpulan operasi (primitif) yang dapat dilakukan terhadap tipe data tersebut tanpa memperhatikan bagaimana implementasi internalnya . Dengan kata lain, *ADT* lebih menekankan pada apa yang dapat dilakukan oleh suatu data, bukan bagaimana data tersebut direpresentasikan secara fisik di dalam memori.

2.1.2 Konsep Kerja Abstract Data Type (ADT)

Konsep kerja ADT bersifat modular, di mana ADT berperan sebagai *supplier* (penyedia) yang menyediakan tipe data dan operasi-operasinya, sedangkan program lain bertindak sebagai *client* (pengguna) yang memanfaatkan ADT tersebut.

a) *Supplier* (penyedia)
   Menyediakan tipe data dan operasi-operasinya
b) *Client* (pengguna)
   Menggunakan *ADT* tanpa perlu mengetahui detail implementasinya

Hal ini memungkinkan pemisahan yang jelas antara spesifikasi dan implementasi, sehingga program menjadi lebih mudah dikembangkan, diuji, dan dipelihara.

Dalam implementasinya, ADT biasanya dibagi menjadi:

1) Spesifikasi
   Berisi definisi tipe data dan operasi
2) *Body*/Implementasi
   Berisi kode program dari operasi
3) Driver Program
   Digunakan untuk menguji *ADT*

2.1.3 Komponen dalam *Abstract Data Type (ADT)*

Sebuah *ADT* umumnya terdiri dari beberapa komponen utama, yaitu:

a) Tipe Data (*Type*)
   Merupakan definisi struktur data yang digunakan, misalnya record atau class yang berisi atribut-atribut tertentu.
b) *Primitif* (Operasi Dasar)Primitif adalah operasi-operasi yang dapat dilakukan terhadap ADT, meliputi:
   1) Konstruktor/Kreator
      Untuk membuat objek.
   2) Selektor (*Getter*)
      Untuk mengambil nilai atribut
   3) Mutator (*Setter*)
      Untuk mengubah nilai atribut

4) Validator

Untuk memastikan data valid

5) Operator Relasional

Untuk membandingkan dua objek

6) Operator Aritmatika

Untuk melakukan perhitungan

7) Konversi

Untuk mengubah tipe data ke bentuk lain

c) *Invariant*

Aturan atau batasan yang harus selalu dipenuhi oleh suatu objek agar tetap valid.

d) Aksioma

Aksioma merupakan aturan atau definisi formal yang menjelaskan hubungan logis antar operasi dalam *ADT*. Aksioma digunakan untuk memastikan bahwa setiap operasi yang dilakukan memberikan hasil yang konsisten dan sesuai dengan spesifikasi.

2.1.4 Primitif Type dalam *Abstract Data Type (ADT)*

Primitif type dalam *ADT* adalah sekumpulan operasi dasar yang didefinisikan untuk memanipulasi tipe data tersebut. Primitif ini biasanya diimplementasikan dalam bentuk fungsi atau prosedur.

Beberapa jenis primitif yang umum digunakan antara lain:

1) *IsValid*

Untuk mengecek validitas data.

2) *Make/Create*

Untuk membentuk objek baru.

3) *Get*

Untuk mengambil nilai komponen.

4) *Set*

Untuk mengubah nilai komponen.

5) *Compare*

   Untuk membandingkan dua objek.

6) Operasi tambahan

   Seperti penjumlahan, pengurangan, atau manipulasi lainnya sesuai kebutuhan.

Primitif-primitif ini menjadi inti dari sebuah *ADT* karena menentukan bagaimana data dapat digunakan dan dimanipulasi dalam program.

2.1.5 *Stack*

Stack merupakan salah satu struktur data linear yang menerapkan prinsip *LIFO (Last In First Out),* yaitu data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan. Struktur ini dapat dianalogikan seperti tumpukan benda, di mana penambahan dan pengambilan data hanya dapat dilakukan pada satu sisi, yaitu bagian atas (*top*).

Operasi dasar pada stack meliputi:

1) *Push*
   Yaitu operasi untuk menambahkan elemen ke dalam *stack*.
2) *Pop*
   Merupakan operasi untuk menghapus elemen teratas dari *stack*.
3) *Peek*
   Operasi untuk melihat elemen teratas tanpa menghapusnya.
4) *IsEmpty*
   Operasi untuk mengecek apakah *stack* dalam keadaan kosong.

Stack banyak digunakan dalam berbagai kasus, seperti evaluasi ekspresi matematika (*postfix*), manajemen memori, serta proses *undo-redo* pada aplikasi.

2.1.6 Implementasi *Stack* dalam *Java*

Dalam bahasa pemrograman *Java, stack* dapat diimplementasikan dengan beberapa cara, antara lain:.

1) Menggunakan *Class Stack (Library Java)*

Java menyediakan *class Stack* dalam package java.util yang memiliki operasi bawaan seperti *push(), pop(),* dan *peek*(). Cara ini lebih praktis karena tidak perlu membuat struktur dari awal.

2) Menggunakan *Array*

Stack dapat diimplementasikan menggunakan *array* dengan bantuan variabel *top* sebagai penunjuk posisi elemen teratas. Pada implementasi ini, perlu diperhatikan kondisi:

a) *Overflow*, yaitu ketika stack penuh.

b) *Underflow*, yaitu ketika stack kosong tetapi dilakukan operasi *pop*.

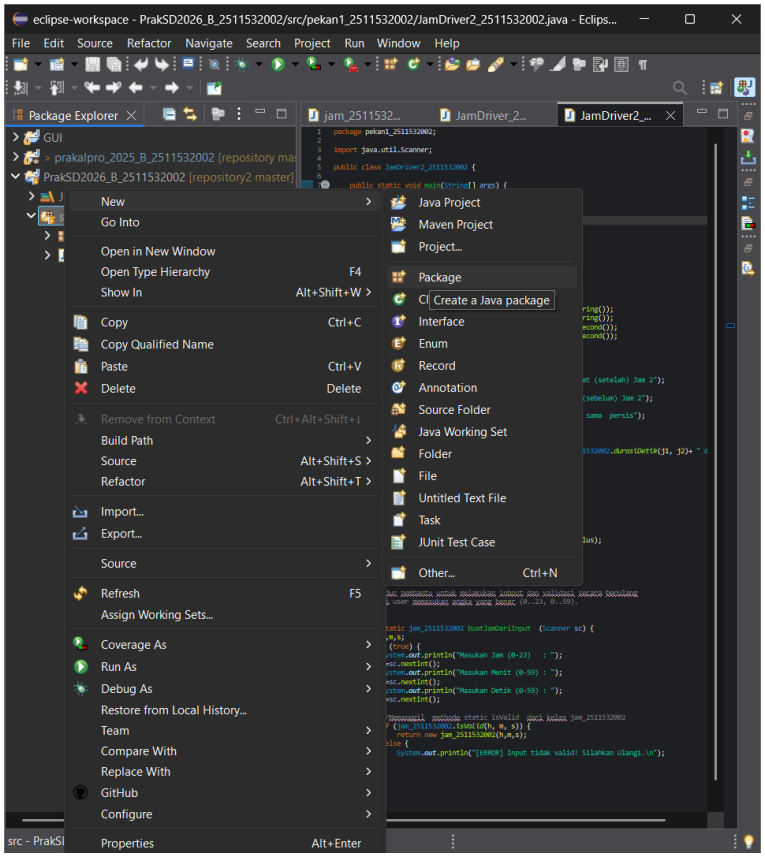
3) Menggunakan *ArrayList*

Selain *array, stack* juga dapat dibuat menggunakan *ArrayList* yang bersifat dinamis. Pendekatan ini lebih fleksibel karena ukuran data dapat berubah secara otomatis sesuai kebutuhan.

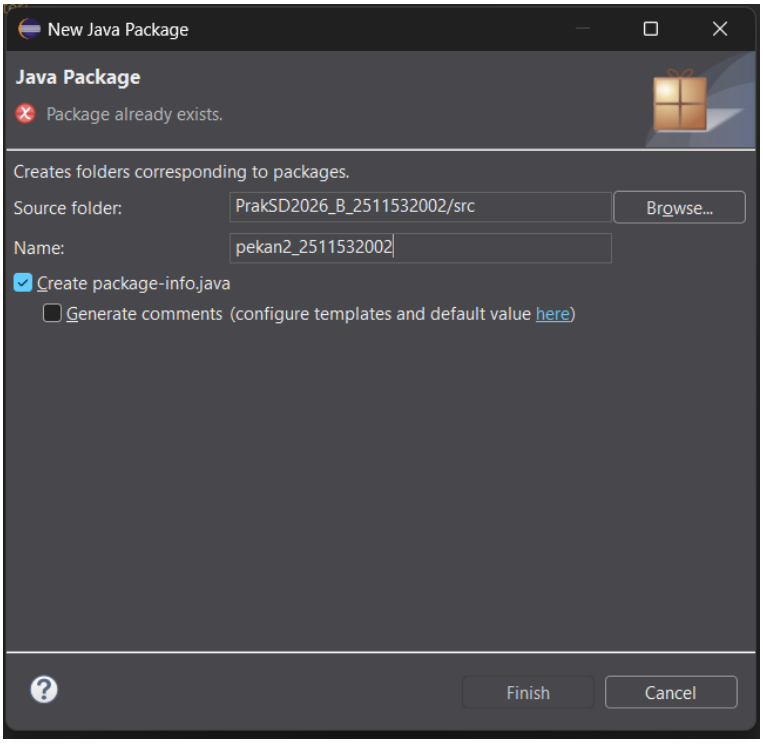
2.2 Langkah Kerja

2.2.1 Pembuatan *Package* dan *Class*

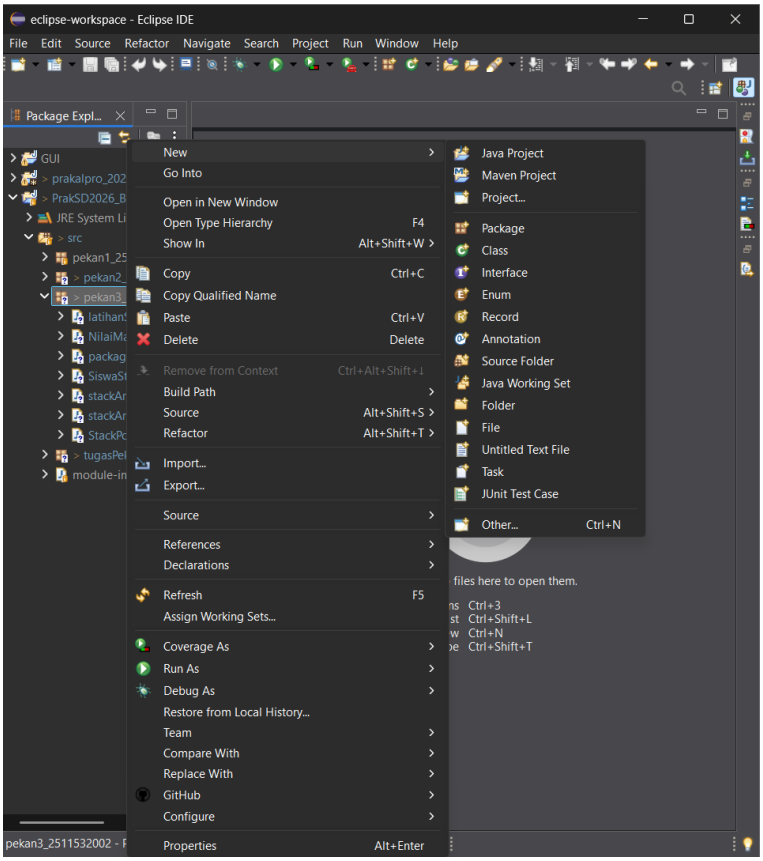
1) Sebelum membuat suatu program *user* perlu untuk membuat *package* terlebih dahulu. Klik kanan pada *src* pada *repository* *"PrakSD2026_B_2511532002"* lalu pilih *"New"* dan klik *"Package"*.



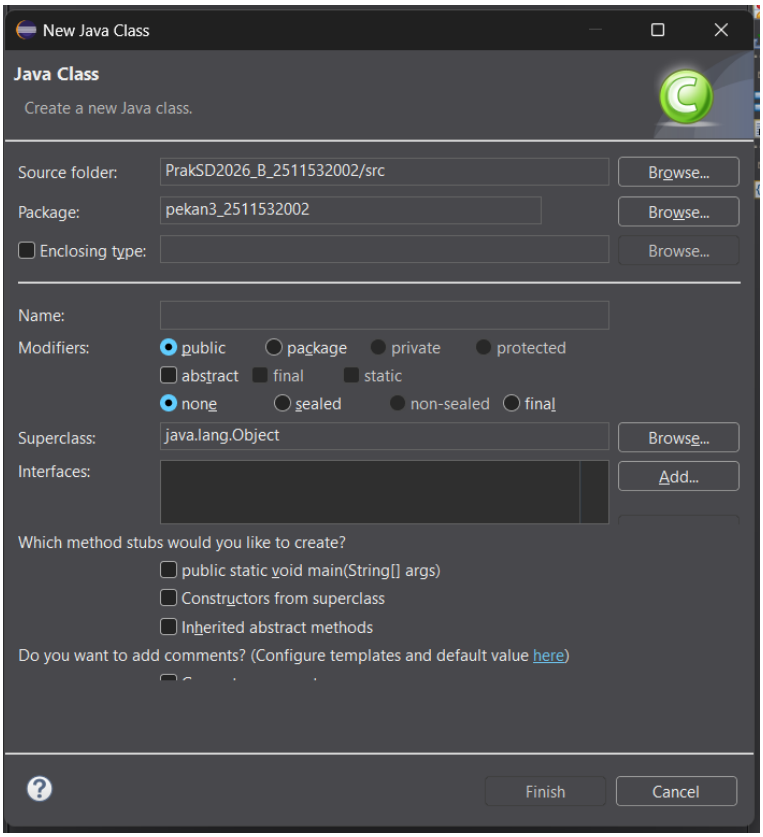
2) Setelah itu akan muncul *pop up* "*Java Package*", buat nama *package* dengan ketentuan tanpa *capslock, space*, dan karakter khusus lainnya. Lalu klik "*finnish*".



3) Selajutnya adalah membuat membuat *class*. pada package yang telah dibuat tapi klik kanan lalu pilih "*New*" dan klik "*Class*".



4) Setelah itu pada menu "*Java Class*" buat nama *class*, dengan ketentuan nama harus capslock diawal kalimat dan tanpa spasi.



2.2.2 Membuat Program *StackArray_2511532002*

stackArray_2511532002.java

```java
package pekan3_2511532002;

public class stackArray_2511532002 {
    static final int MAX = 1000;
    int top;
    int a[]=new int [MAX];
    boolean isEmpty()
    {
        return (top <0);
    }
    stackArray_2511532002()
    {
        top=-1;
    }
    boolean push_2511532002(int x) {
        if (top>= (MAX-1)) {
            System.out.println("Stack Overflow");
            return false;
        }else {
            a[++top]=x;
            System.out.println(x + " dimasukan dalam stack");
            return true;
        }
    }
    int pop_2511532002() {
        if (top<0) {
            System.out.println("Stack Underflow");
            return 0;
        } else {
        int x =a[top--];
        return x;
    }
}
    int peek_2511532002() {
        if (top < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }else {
            int x=a[top];
            return x;
        }
    }
    void print_2511532002() {
        for (int i = top; i >-1; i--) {
            System.out.println(" "+a[i]);
        }
    }
}
```

Setelah membuat *class StackArray_2511532002,* maka mahasiswa dapat menulis *syntax* nya.

1. Menentukan atribut (*Type*) yang digunakan dalam *Stack*.
   Static final int MAX_2002 = 1000;
   Int top_2002;
   Int a_2002[] = new int [MAX_2002];

2. Membuat konstruktor untuk menginisialisasi *Stack*.
   stackArray_2511532002(){
   top_2002 = -1;
   }

3. Membuat *methode* untuk mengecek apakah *Stack* kosong.
   boolean isEmpty_2002() {
   return (top_2002 < 0);
   }

4. Membuat *methode* untuk menambahkan data ke dalam *Stack (push)*.

```
boolean push_2002(int x) {
    if (top_2002 >= (MAX_2002 – 1)) {
            System.out.println("Stack Overflow");
            return false;
    }
  else {
    a_2002[++top_2002] = x;
            System.out.println(x + " dimasukkan dalam stack");
            return true;
    }
}
```

5. Membuat *methode* untuk menghapus data dari *Stack (pop)*.

```
Int pop_2002() {
    if (top_2002 < 0) {


            System.out.println("Stack Underflow");
            return 0;
    }
  else {
            int x = a_2002[top_2002--];
            return x;
    }
}
```

6. Membuat *methode* untuk melihat elemen teratas *(peek)*.

```
Int pop_2002() {
    if (top_2002 < 0) {
            System.out.println("Stack Underflow");
            return 0;
    }
  else {
            int x = a_2002[top_2002];
            return x;
    }
}
```

7. Membuat *methode* untuk menampilkan isi *Stack.*

```
void print_2002(){
    for (int I = top_2002; I > -1; i--) {
            System.out.print(" " + a_2002[i]);
    }
}
```

2.2.3 Membuat Program *StackArrayDriver*_2511532002

```
stackArray_2511532002.java    stackArrayDriver_2511532002.java
1  package pekan3_2511532002;
2
3  public class stackArrayDriver_2511532002 {
4
5      public static void main(String[] args) {
6          stackArray_2511532002 s = new stackArray_2511532002();
7          s.push_2511532002(10);
8          s.push_2511532002(20);
9          s.push_2511532002(30);
10         System.out.println(s.pop_2511532002()+" dikeluarkan dari stack");
11         System.out.println("Elemen teratas adalah : "+s.peek_2511532002());
12         System.out.println("Elemen pada stack : " );
13         s.print_2511532002();
14     }
15 }

Console
<terminated> stackArrayDriver_2511532002 [Java Application] C:\Users\user\.p2\pool\plugins\
10 dimasukan dalam stack
20 dimasukan dalam stack
30 dimasukan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah : 20
Elemen pada stack :
 20
 10
```

Membuat *class driver* sebagai program utama. *Class* ini digunakan sebagai tempat menjalankan program *Stack.*

1. Membuat *methode* main sebagai titik awal eksekusi program.
   public static void main(String[] args) {

2. Membuat objek dari *class Stack Array.* Digunakan dalam program.
   stackArray_2511532002 s = new stackArray_2511532002();

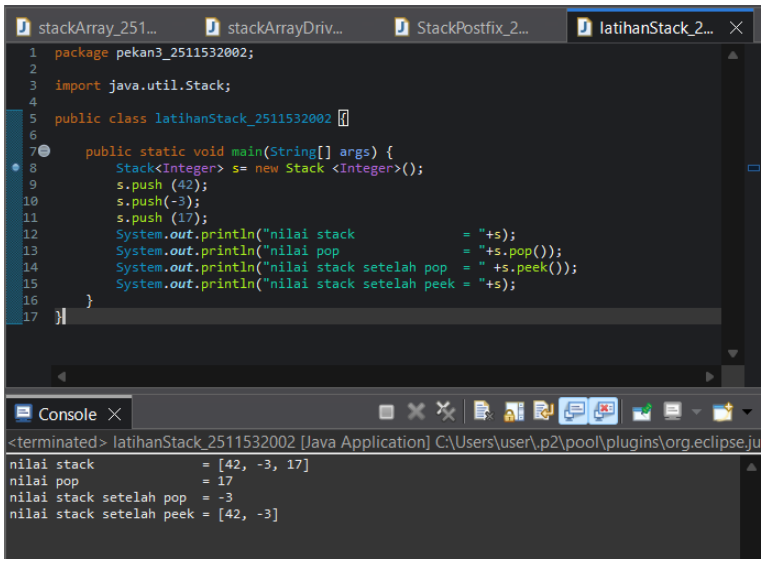
3. Menambahkan data ke dalam *Stack (push).* digunakan untuk memasukkan beberapa elemen ke dalam *Stack.*
   s.push_2002(10);
   s.push_2002(20);
   s.push_2002(30);

4. Menghapus elemen teratas dari *Stack (pop).* Digunakan untuk mengambil dan menghapus elemen paling atas dari *Stack,* kemudian menampilkannya.
   System.out.println(s.pop_2002() + " dikeluarkan dari stack");

5. Menampilkan elemen teratas *(peek).* Digunakan untuk melihat elemen paling atas tanpa menghapusnya.
   System.out.println("Elemen teratas adalah : " + s.peek_2002());

6. Menampilkan seluruh isi *stack.* Digunakan untuk menampilkan semua elemen yang masih tersisa dalam *stack.*
   System.out.print("Elemen pada stack :");
   s.print_2002();

2.2.4 Membuat Program latihanStack_2511532002



1. Membuat *method main* sebagai awal untuk eksekusi program.
   public static void main(String[] args) {

2. Membuat objek *Stack* dari *library Java*, untuk menyimpan data bertipe *Integer* dengan konsep *LIFO*.
   Stack<Integer> s = new Stack<Integer>();

3. Menambahkan data ke dalam *Stack* dengan *push*.
   s.push(42);
   s.push(-3);
   s.push(17);

4. Menampilkan seluruh isi *stack*.
   System.out.println("nilai stack = " + s);

5. Menghapus elemen teratas *dari Stack* dengan *pop.* Digunakan untuk mengambil dan menghapus elemen paling atas dari stack, kemudian menampilkannya.
   System.out.println("nilai pop = " + s.pop());

6. Menampilkan elemen teratas setelah pop (peek) dan selanjutnya menampilkan kembali hasil setelah operasi.
   System.out.println("nilai stack setelah pop = " + s.peek());
   System.out.println("nilai stack setelah peek = +s");

2.2.5 Membuat Program NilaiMaksimum_2511532002

```
NilaiMaksim...   stackArrayD...   StackPostfix...   latihanStac...

1  package pekan3_2511532002;
2
3  import java.util.Stack;
4
5  public class NilaiMaksimum {
6      public static int max(Stack<Integer>s) {
7          Stack <Integer> backup_2511532002 = new Stack <Integer>();
8          int maxValue_2511532002 = s.pop();
9          backup_2511532002.push(maxValue_2511532002);
10         while (!s.isEmpty()) {
11             int next_2511532002 = s.pop();
12             backup_2511532002.push(next_2511532002);
13             maxValue_2511532002=Math.max(maxValue_2511532002, next_2511532002);
14         }
15         while (!backup_2511532002.isEmpty()) {
16             s.push(backup_2511532002.pop());
17         }
18         return maxValue_2511532002;
19     }
20     public static void main(String[] args) {
21         Stack<Integer> s =new Stack<Integer>();
22         s.push(70);
23         s.push(12);
24         s.push(20);
25         System.out.println("Isi Stack      : "+s);
26         System.out.println("Stack Teratas  : "+s.peek());
27         System.out.println("Nilai Maksimum : "+max(s));
28     }
29 }
30

Console
<terminated> NilaiMaksimum [Java Application] C:\Users\user\.p2\pool\plugins\org.eclipse.justj.openjd
Isi Stack      : [70, 12, 20]
Stack Teratas  : 20
Nilai Maksimum : 70
```

1. Membuat *method main* sebagai titik awal eksekusi program.
   public static void main(String[] args) {

2. Membuat *method max* untuk mencari nilai maksimum dalam *stack* tanpa menghilangkan isi *stack*.
   public static int max(Stack<Integer> s) {

3. Membuat *stack* bantuan (*backup*) untuk menyimpan sementara elemen agar data asli tidak hilang.
   Stack<Integer> backup_2511532002 = new Stack<Integer>();

4. Mengambil elemen pertama sebagai nilai awal maksimum. Elemen teratas di-*pop* dan dijadikan nilai awal untuk dibandingkan.
   int maxValue_2511532002 = s.pop();
   backup_2511532002.push(maxValue_2511532002);

5. Melakukan perulangan untuk mencari nilai maksimum. Dimana setiap elemen di-*pop*, dibandingkan, lalu disimpan ke *stack backup.*
```
while (!s.isEmpty()) {
int next_2511532002 = s.pop();
backup_2511532002.push(next_2511532002);
maxValue_2511532002 = Math.max(maxValue_2511532002,
next_2511532002);
}
```

6. Mengembalikan isi *stack* ke kondisi semula.
Elemen dari stack *backup* dikembalikan lagi ke *stack* awal.
```
while (!backup_2511532002.isEmpty()) {
s.push(backup_2511532002.pop());
}
```

7. Mengembalikan nilai maksimum.
```
return maxValue_2511532002;
```

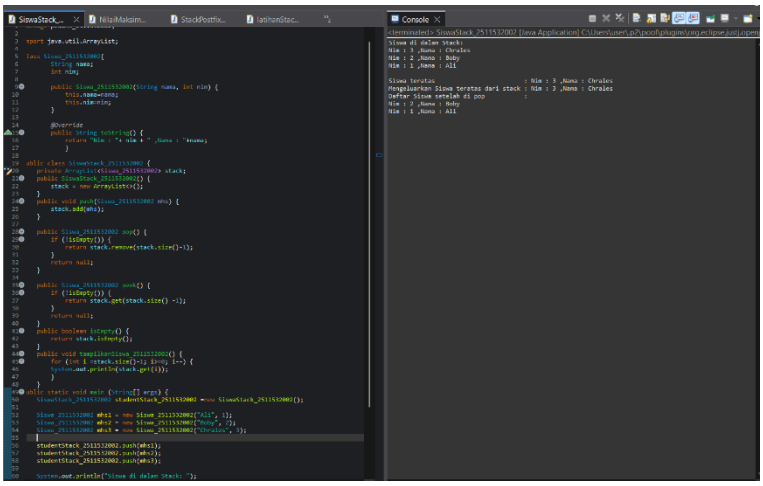
8. Membuat objek *Stack* dan menambahkan data, untuk mengisi *stack* dengan beberapa nilai.
```
Stack<Integer> s = new Stack<Integer>();
s.push(70);
s.push(12);
s.push(20);
```

9. Menampilkan isi stack.
```
System.out.println("Isi Stack      : " + s);
```

10. Menampilkan elemen teratas (*peek*), untuk melihat elemen paling atas tanpa menghapusnya.
```
System.out.println("Stack Teratas  : " + s.peek());
```

11. Menampilkan nilai maksimun dengan memanggil *method max*() untuk mendapatkan nilai terbesar dalam *stack*.
```
System.out.println("Nilai Maksimum : " + max(s));
```

2.2.6 SiswaStack_2511532002



1. Membuat *constructor* pada *class Siswa*, untuk menginisialisasi nilai atribut saat objek dibuat.
   public Siswa_2511532002(String nama, int nim) {
   this.nama = nama;
   this.nim = nim;
2. Menampilkan data siswa dalam bentuk teks saat dipanggil.
   @Override
   public String toString() {
   return "Nim : "+ nim + " ,Nama : "+nama;
   }

3. Membuat struktur *stack* menggunakan *ArrayList* untuk tempat penyimpanan data siswa secara dinamis.
   private ArrayList<Siswa_2511532002> stack;

4. Membuat *constructor stack*, untuk menginisialisasi *ArrayList* sebagai *stack.*
   public SiswaStack_2511532002() {
   stack = new ArrayList<>();
   }

5. Membuat *method push*, untuk menambahkan data siswa ke dalam *stack.*
   public void push(Siswa_2511532002 mhs) {
   stack.add(mhs);
   }

6. Membuat *method pop*, untuk mengambil dan menghapus elemen teratas dari stack.
```
public Siswa_2511532002 pop() {
if (!isEmpty()) {
return stack.remove(stack.size() - 1);
}
return null;
}
```

7. Membuat *method peek*, untuk melihat elemen teratas tanpa menghapusnya.
```
public Siswa_2511532002 peek() {
if (!isEmpty()) {
return stack.get(stack.size() - 1);
}
return null;
}
```

8. Membuat method *isEmpty*, untuk mengecek apakah stack kosong.
```
public boolean isEmpty() {
return stack.isEmpty();
}
```
9. Membuat method *tampilkanSiswa*, untuk menampilkan seluruh data siswa dalam stack menggunakan perulangan dari atas ke bawah (*LIFO*).
```
public void tampilkanSiswa_2511532002() {
for (int i = stack.size() - 1; i >= 0; i--) {
System.out.println(stack.get(i));
    }
}
```

10. Membuat objek *stack* dan data siswa, untuk membuat beberapa objek siswa.
```
SiswaStack_2511532002 studentStack_2511532002 = new SiswaStack_2511532002();
Siswa_2511532002 mhs1 = new Siswa_2511532002("Ali", 1);
Siswa_2511532002 mhs2 = new Siswa_2511532002("Boby",2);
Siswa_2511532002 mhs3 = new Siswa_2511532002("Chrales",3);
```

11. Menambahkan data ke dalam stack.
```
studentStack_2511532002.push(mhs1);
studentStack_2511532002.push(mhs2);
studentStack_2511532002.push(mhs3);
```

12. Menampilkan isi stack.
    studentStack_2511532002.tampilkanSiswa_2511532002();

13. Menampilkan elemen teratas (*peek*).
    System.out.println("Siswa teratas : " + studentStack_2511532002.peek());

14. Menghapus elemen teratas (pop).
    System.out.println("Mengeluarkan Siswa teratas dari stack : " + studentStack_2511532002.pop());

15. Menampilkan isi stack setelah pop.
    studentStack_2511532002.tampilkanSiswa_2511532002();

2.2.7 Membuat Program *StackPostfix_2511532002*

stackArray_2511532002.java | stackArrayDriver_2511532002.java | StackPostfix_2511532002.java

```
1  package pekan3_2511532002;
2
3  import java.util.Scanner;
4  import java.util.Stack;
5
6  public class StackPostfix_2511532002 {
7      public static int postfixEvaluate(String expression) {
8          Stack<Integer> s = new Stack<Integer>();
9          Scanner input_2511532002 = new Scanner (expression);
10         while (input_2511532002.hasNext()) {
11             if (input_2511532002.hasNextInt()) {            //an operand (int)
12                 s.push (input_2511532002.nextInt());        //an operators
13             }else {
14                 String operator=input_2511532002.next();
15                 int operand2 = s.pop();
16                 int operand1 =s.pop();
17                 if (operator.equals("+")) {
18                     s.push(operand1+operand2);
19                 }else if (operator.equals("-")) {
20                     s.push(operand1-operand2);
21                 }else if (operator.equals("*")) {
22                     s.push(operand1*operand2);
23                 }else {
24                     s.push(operand1/operand2);
25                 }
26             }
27         }
28         input_2511532002.close();
29         return s.pop();
30     } public static void main (String[] args) {
31         System.out.println("Hasil Postfix = "+postfixEvaluate("5 2 4 * + 7 -"));
32     }
33 }
```

1. Membuat *class StackPostfix_2511532002. Class* ini digunakan untuk mengimplementasikan evaluasi ekspresi *postfix.*
   public class StackPostfix_2002 {

2. Membuat *methode postfixEvaluate. Methode* ini digunakan untuk menghitung hasil dari ekspresi *postfix.*
   public static int postfixEvaluate(String expression) {

3. Membuat *Stack* dan *Scanner. Stack* digunakan untuk menyimpan *operand,* sedangkan *Scanner* untuk membaca input.
   Stack<Integer> s_2002 = new Stack<Integer>();
   Scanner input_2002 = new Scanner (expression);

4. Membaca setiap elemen dalam ekspresi. Digunakan untuk memproses seluruh input selama masih ada data.
   While (input_2002.hasNext()) {

5. Mengecek apakah input berupa angka (*operand)*. Jika berupa, maka langsung dimasukkan ke dalam *Stack.*
   If (input_2002.hasNextInt()) {
   s_2002.push (input_2002.nextInt());
   }

6. Memproses operator. Jika bukan angka, maka dianggap sebagai operator dan dilakukan operasi terhadap dua elemen teratas.
   String operator = input_2002.next();
   int operand2 = s_2002.pop();
   int operand1 = s_2002.pop();

7. Melakukan operasi aritmatika. Digunakan untuk menghitung hasil berdasarkan operator.
   If (operator.equals("+")) {
   s_2002.push(operand1 + operand2);
   } else if (operator.equals("-")) {
   s_2002.push(operand1 – operand2);
   } else if (operator.equals("")) {*
   *s_2002.push(operand1 * operand2);*
   } else {
   s_2002.push(operand1 / operand2);
   }

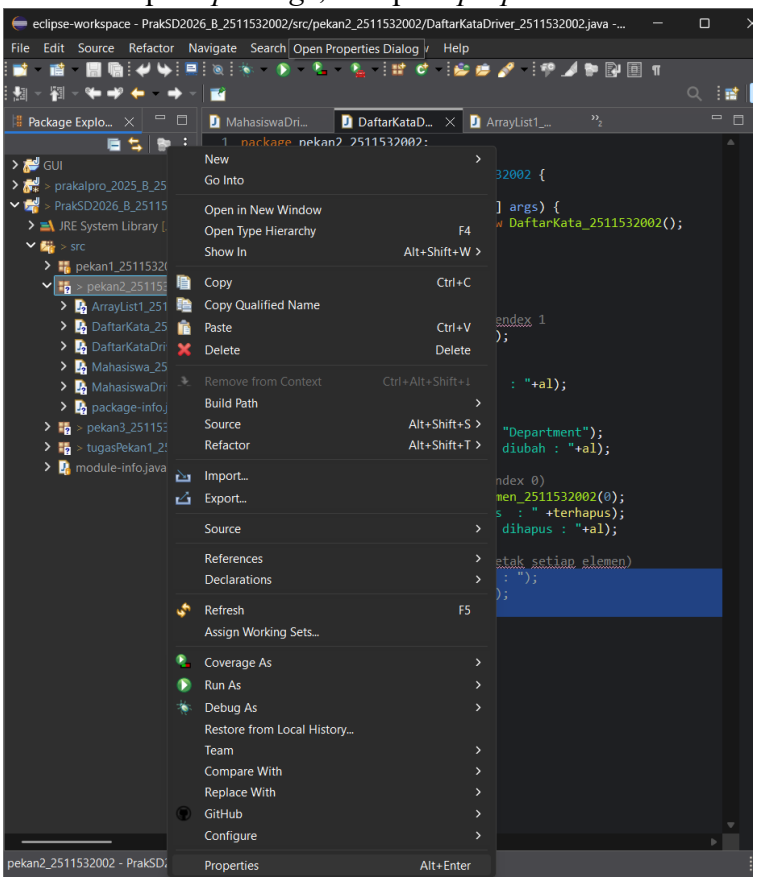
8. Menutup *Scanner* dan mengembalikan hasil. Setelah proses selesai, hasil akhir diambil dari *stack.*
   input_2002.close();
   return s_2002.pop();

9. Membuat *methode main* untuk menjalankan program. Digunakan untuk menampilkan hasil evaluasi *postfix.*

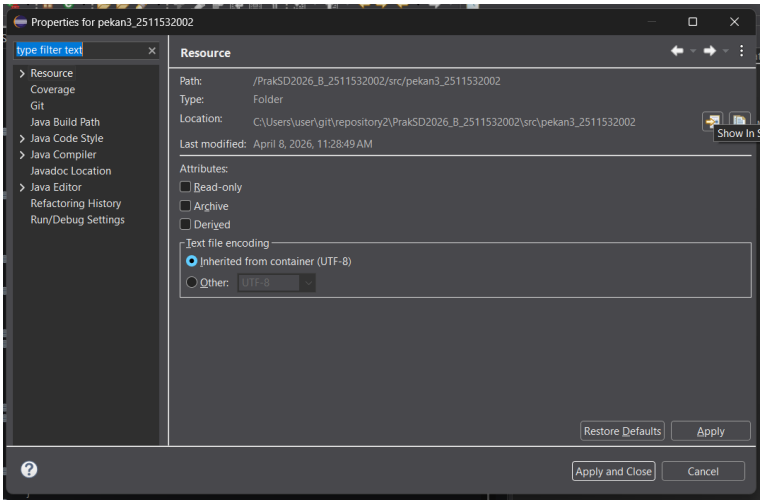
   public static void main(String[] args) {
   *System.out.println("hasil postfix= " + postfixEvaluate("5 2 4 * + 7 –"));*
   }

2.2.8 *Upload* ke GitHub

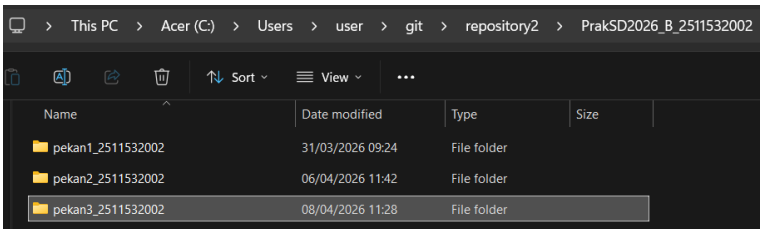
1. Setelah semua program selesai dibuat, selanjutnya adalah *user* perlu untuk memasukan program yang dibuat di Eclips ke *repository* GitHub. Selain dengan cara *"Team"* atau *"Commit and Push"*. Bisa juga dengan cara *upload* manual. Caranya klik kanan pada *package,* lalu pilih *properties.*



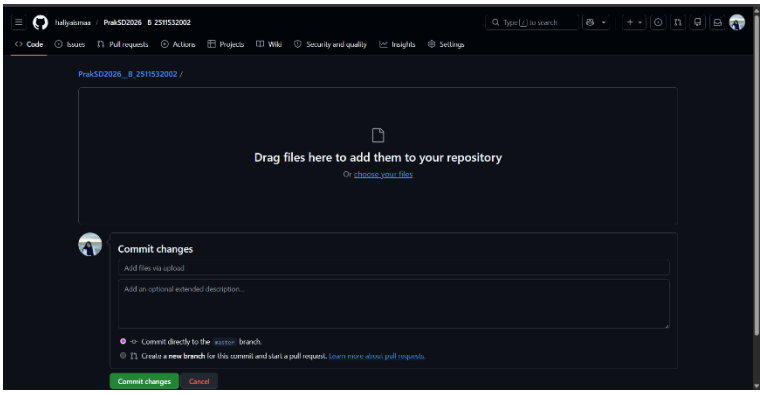
2. Maka akan muncul *pop up menu "Properteis for pekan3_2511532002"*. Lalu klik *icon "Show in System Folder"*.



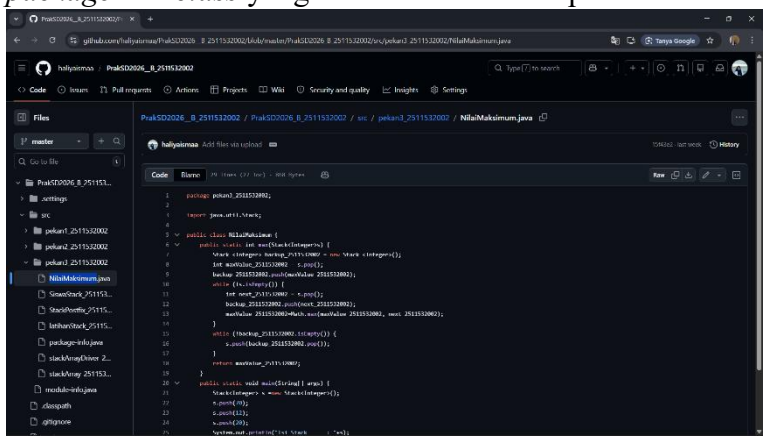
3. Maka akan menunjukan lokasi *file "pekan3_2511532002"* pada *File Explorer computer.*



4. Setelah itu *drag and drop file* "*pekan1_2511532002*" ke GitHub. Lalu tambahkan keterangan jika perlu. Setelah itu klik *upload.*

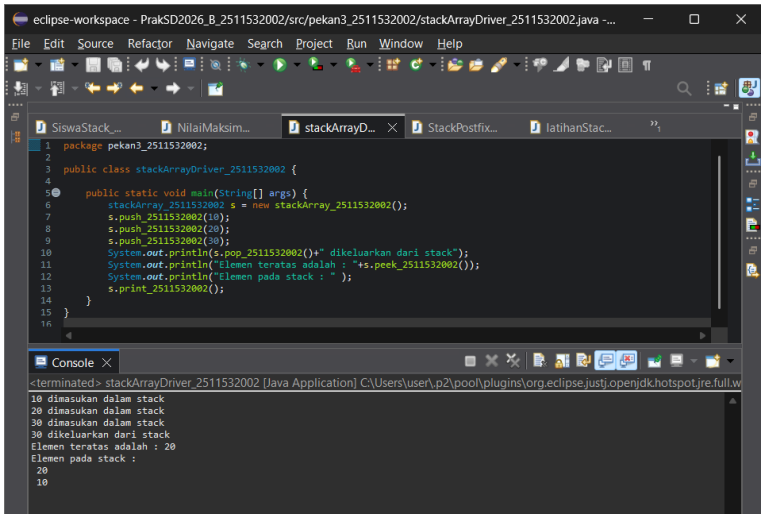


5. Maka dapat dilihat pada *repository GitHub* bahwa file "*pekan3_2511532002*" telah terunggah via *Upload.*
Dapat kita lihat disini jika kita buka, maka akan muncul *file package* dan *class* yang telah di buat di Eclipse.



2.3 Hasil

2.3.1 Hasil *StackArrayDriver*_2511532002



2.3.2 Hasil *latihanStack_2511532002*

```java
package pekan3_2511532002;

import java.util.Stack;

public class latihanStack_2511532002 {

    public static void main(String[] args) {
        Stack<Integer> s= new Stack <Integer>();
        s.push (42);
        s.push(-3);
        s.push (17);
        System.out.println("nilai stack              = "+s);
        System.out.println("nilai pop                = "+s.pop());
        System.out.println("nilai stack setelah pop  = " +s.peek());
        System.out.println("nilai stack setelah peek = "+s);
    }
}
```

```
<terminated> latihanStack_2511532002 [Java Application] C:\Users\user\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32
nilai stack              = [42, -3, 17]
nilai pop                = 17
nilai stack setelah pop  = -3
nilai stack setelah peek = [42, -3]
```

2.3.3 Hasil *NilaiMaksimum_2511532002*

```java
package pekan3_2511532002;

import java.util.Stack;

public class NilaiMaksimum {
    public static int max(Stack<Integer>s) {
        Stack <Integer> backup_2511532002 = new Stack <Integer>();
        int maxValue_2511532002 = s.pop();
        backup_2511532002.push(maxValue_2511532002);
        while (!s.isEmpty()) {
            int next_2511532002 = s.pop();
            backup_2511532002.push(next_2511532002);
            maxValue_2511532002=Math.max(maxValue_2511532002, next_2511532002);
        }
        while (!backup_2511532002.isEmpty()) {
            s.push(backup_2511532002.pop());
        }
        return maxValue_2511532002;
    }
    public static void main(String[] args) {
        Stack<Integer> s =new Stack<Integer>();
        s.push(70);
        s.push(12);
        s.push(20);
        System.out.println("Isi Stack      : "+s);
        System.out.println("Stack Teratas  : "+s.peek());
        System.out.println("Nilai Maksimum : "+max(s));
    }
}
```

```
<terminated> NilaiMaksimum [Java Application] C:\Users\user\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_2
Isi Stack      : [70, 12, 20]
Stack Teratas  : 20
Nilai Maksimum : 70
```

2.3.4 Hasil *SiswaStack_251152002*

```java
package pekan3_2511532002;

import java.util.ArrayList;

class Siswa_2511532002{
        String nama;
        int nim;

        public Siswa_2511532002(String nama, int nim) {
            this.nama=nama;
            this.nim=nim;
        }

        @Override
        public String toString() {
            return "Nim : "+ nim + " ,Nama : "+nama;
            }
}
public class SiswaStack_2511532002 {
    private ArrayList<Siswa_2511532002> stack;
    public SiswaStack_2511532002() {
        stack = new ArrayList<>();
    }
    public void push(Siswa_2511532002 mhs) {
        stack.add(mhs);
    }

    public Siswa_2511532002 pop() {
        if (!isEmpty()) {
```

```
<terminated> SiswaStack_2511532002 [Java Application] C:\Users\user\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32
Siswa di dalam Stack:
Nim : 3 ,Nama : Chrales
Nim : 2 ,Nama : Boby
Nim : 1 ,Nama : Ali

Siswa teratas                       : Nim : 3 ,Nama : Chrales
Mengeluarkan Siswa teratas dari stack : Nim : 3 ,Nama : Chrales
Daftar Siswa setelah di pop         :
Nim : 2 ,Nama : Boby
Nim : 1 ,Nama : Ali
```

2.3.5 Hasil *StackPostfix_2511532002*

SiswaStack_... NilaiMaksim... stackArrayD... StackPostfix... latihanStac...

```java
package pekan3_2511532002;

import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2511532002 {
    public static int postfixEvaluate(String expression) {
        Stack<Integer> s = new Stack<Integer>();
        Scanner input_2511532002 = new Scanner (expression);
        while (input_2511532002.hasNext()) {
            if (input_2511532002.hasNextInt()) {            //an operand (int)
                s.push (input_2511532002.nextInt());        //an operators
            }else {
                String operator=input_2511532002.next();
                int operand2 = s.pop();
                int operand1 =s.pop();
                if (operator.equals("+")) {
                    s.push(operand1+operand2);
                }else if (operator.equals("-")) {
                    s.push(operand1-operand2);
                }else if (operator.equals("*")) {
                    s.push(operand1*operand2);
                }else {
                    s.push(operand1/operand2);
                }
            }
        }
        input_2511532002.close();
        return s.pop();
    } public static void main (String[] args) {
        System.out.println("Hasil Postfix = "+postfixEvaluate("5 2 4 * + 7 -"));
    }
}
```

Console

```
<terminated> StackPostfix_2511532002 [Java Application] C:\Users\user\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32
Hasil Postfix = 6
```

BAB III
KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep *Abstract Data Type (ADT*) dapat diterapkan dengan baik dalam pemrograman *Java*, khususnya pada implementasi struktur data stack. Stack merupakan struktur data linear yang menggunakan prinsip *LIFO (Last In First Out),* di mana elemen terakhir yang masuk akan menjadi elemen pertama yang keluar.

Implementasi *stack* dapat dilakukan dengan berbagai pendekatan, seperti menggunakan *class Stack* dari *library Java, array*, dan *ArrayList*. Masing-masing pendekatan memiliki kelebihan, di mana penggunaan *library* lebih praktis, sedangkan *array* dan *ArrayList* memberikan pemahaman lebih mendalam terkait cara kerja stack secara manual.

Selain itu, stack juga dapat diterapkan dalam berbagai kasus, seperti mencari nilai maksimum dan evaluasi ekspresi *postfix*, yang menunjukkan bahwa *stack* memiliki peran penting dalam penyelesaian masalah dalam pemrograman. Penggunaan *stack* dengan tipe data sederhana maupun *object* (seperti data siswa) juga membuktikan fleksibilitas struktur data ini dalam berbagai kebutuhan.

3.2 Saran

Saran untuk praktikum selanjutnya adalah agar mahasiswa lebih teliti dalam memahami penulisan *syntax* pemrograman Java dan penggunaan struktur data untuk menghindari kesalahan atau error, terutama dalam penggunaan indeks, pemanggilan *metShod*, serta pengelolaan data dalam program.

DAFTAR PUSTAKA

[1] Inggriani Liem, “Diktat Struktur Data,” Program Studi Teknik Informatika, Institut Teknologi Bandung, 2008.